

JOBCONNECT

Plan integral de trabajo y división para 5 compañeros

Segundo Mini Proyecto · Frontend

Sistema para empresa de empleabilidad · Consumo de servicios con Fetch API

Documento elaborado a partir del enunciado oficial de JobConnect. El proyecto es únicamente frontend, utiliza DummyJSON como API pública, requiere login con token, seis CRUD, Git y el uso obligatorio de NotebookLM.

Elemento	Plan
Equipo	5 compañeros
Duración indicada	3 sesiones: planificación, desarrollo y cierre
Tecnologías	HTML, CSS y JavaScript; Fetch API; async/await
API	DummyJSON
Autenticación	/auth/login + token en localStorage
Módulos	6 CRUD: Candidatos, Vacantes, Empresas, Postulaciones, Entrevistas/Notas y Tareas
Control de versiones	Git obligatorio; ramas y commits frecuentes
Apoyo obligatorio	NotebookLM para investigación, consultas, video e infografía

Objetivo del documento

Organizar el proyecto completo antes de programar, asignando responsabilidades claras, dependencias, archivos, fases, estrategia Git, pruebas y entregables para que los cinco integrantes puedan trabajar en paralelo sin bloquearse ni pisarse código.

1. Alcance completo del proyecto

El sistema debe permitir que un reclutador inicie sesión y administre seis módulos del negocio desde un panel web. El consumo de datos se realiza contra DummyJSON mediante fetch. La autenticación debe controlar el acceso a los módulos.

Requerimientos funcionales que deben cubrirse

- RF-01: iniciar sesión contra /auth/login.
- RF-02: almacenar el token y utilizarlo mediante el header Authorization cuando corresponda.
- RF-03: bloquear el acceso a los módulos si no existe un token válido.
- RF-04: cerrar sesión eliminando el token.
- RF-05: listar registros de los seis módulos con GET.
- RF-06: crear registros con POST.
- RF-07: editar con PUT y/o PATCH según el módulo.
- RF-08: eliminar con DELETE.
- RF-09: mostrar retroalimentación de operaciones exitosas o fallidas.
- RF-10: navegar entre los seis módulos desde una interfaz principal.

Requerimientos de calidad que deben verificarse

- Solo frontend; HTML, CSS y JavaScript.
- Código organizado por carpetas con estructura clara.
- Consumo mediante fetch y async/await.
- Interfaz intuitiva, usable y responsive.
- Errores controlados con try/catch sin romper la aplicación.
- Git con repositorio remoto, ramas y commits descriptivos/frecuentes.
- Código legible, nombres significativos e indentación consistente.
- Credenciales y tokens no deben quedar expuestos de forma insegura.
- README.md completo.

Nota crítica: DummyJSON simula POST/PUT/PATCH/DELETE y devuelve respuestas, pero no persiste los cambios en su servidor. Por tanto, una recarga puede volver a mostrar los datos originales; esto forma parte del comportamiento esperado de la API.

2. División de los cinco compañeros

La división busca equilibrar la carga: tres integrantes reciben dos CRUD cada uno (dos módulos por persona); el primero se concentra en autenticación, arquitectura e integración; el quinto en UI, QA y entregables, participando también en la documentación técnica.

Integrante	Responsabilidad principal	Módulos / área	Resultado
C1 · Líder	Arquitectura, autenticación, servicios, integración y UI	Integración y UI	Base funcional e integración final
C2	CRUD de datos	Candidatos (/users) + Vacantes (/products)	2 CRUD completos
C3	CRUD de datos	Empresas (/carts) + Postulaciones (/posts)	2 CRUD completos
C4	CRUD de datos	Entrevistas/Notas (/comments) + Tareas (/todos)	2 CRUD completos
C5 · QA/UI	Interfaz, pruebas, documentación y evidencias	UI global + QA + README/bitácora/videos/interfaz	Entregables

Responsabilidades detalladas

C1 · Líder / Integración

- Crear estructura inicial y convenciones.
- Implementar login, logout, token y protección de acceso.
- Crear servicio API compartido para evitar duplicación.
- Definir estrategia de ramas y revisar integraciones.
- Mantener la rama de integración y coordinar la entrega final.

C2 · Candidatos + Vacantes

- Implementar GET, POST, PUT, PATCH y DELETE en /users.
- Implementar GET, POST, PUT, PATCH y DELETE en /products.
- Crear tablas/listados, formularios, edición, eliminación y mensajes.
- Entregar una breve guía de datos/campos usados por cada recurso.

C3 · Empresas + Postulaciones

- Implementar GET, POST, PUT y DELETE en /carts.
- Implementar GET, POST, PATCH y DELETE en /posts.
- Crear sus vistas, formularios y estados de carga/error.
- Documentar decisiones de mapeo del recurso al dominio.

C4 · Entrevistas + Tareas

- Implementar GET, POST, PATCH y DELETE en /comments.
- Implementar GET, POST, PATCH y DELETE en /todos.
- Crear vistas, formularios, edición parcial y eliminación.
- Documentar campos y comportamiento de cada módulo.

C5 · UI + QA + documentación

- Construir y normalizar estilos del login, dashboard, tablas, formularios y mensajes.
- Verificar responsive y consistencia visual.

- Diseñar matriz de pruebas y ejecutar pruebas de integración.
- Recopilar evidencias, bitácora NotebookLM y material de presentación.
- Coordinar con C1 la versión final del README.

3. Matriz de módulos y métodos HTTP

Esta tabla debe usarse como lista de control durante el desarrollo. No se deben agregar métodos no solicitados como sustitutos de los indicados.

#	Módulo de negocio	DummyJSON	GET	POST	PUT	PATCH	DELETE
1	Candidatos	/users	✓	✓	✓	✓	✓
2	Vacantes	/products	✓	✓	✓	✓	✓
3	Empresas clientes	/carts	✓	✓	✓	—	✓
4	Postulaciones	/posts	✓	✓	—	✓	✓
5	Entrevistas / notas	/comments	✓	✓	—	✓	✓
6	Tareas del reclutador	/todos	✓	✓	—	✓	✓

Regla de implementación

Cada módulo debe reutilizar, cuando sea posible, el servicio de API compartido de C1. Así se evita que cada compañero cree su propia configuración de base URL, headers, token y manejo de errores.

Autenticación

Paso	Comportamiento esperado
1	Usuario introduce emilys / emilyspass.
2	Frontend hace POST a /auth/login.
3	Se obtiene el token.
4	Token se guarda en localStorage.
5	El usuario entra al dashboard.
6	Las solicitudes protegidas usan Authorization cuando corresponda.
7	Si no existe token, se impide el acceso a módulos.
8	Logout elimina el token y devuelve al login.

4. Arquitectura y estructura de archivos

Se propone una estructura que minimiza conflictos. Cada compañero tendrá principalmente archivos propios y solo habrá archivos compartidos cuidadosamente definidos.

```
JobConnect/
├─ index.html
├─ README.md
├─ pages/
│  ├─ login.html
│  ├─ dashboard.html
│  ├─ candidatos.html
│  ├─ vacantes.html
│  ├─ empresas.html
│  ├─ postulaciones.html
│  ├─ entrevistas.html
│  └─ tareas.html
├─ css/
│  ├─ style.css
│  ├─ login.css
│  ├─ dashboard.css
│  └─ responsive.css
├─ js/
│  ├─ app.js
│  ├─ auth.js
│  ├─ candidatos.js
│  ├─ vacantes.js
│  ├─ empresas.js
│  ├─ postulaciones.js
│  ├─ entrevistas.js
│  └─ tareas.js
├─ services/
│  ├─ api.js
│  └─ auth.js
├─ assets/
│  └─ ...
├─ docs/
├─ planificacion.md
├─ bitacora-notebooklm.md
└─ pruebas.md
```

Propiedad recomendada de archivos

Archivo / carpeta	Principal	Regla
services/api.js	C1	No modificar sin coordinar; servicio compartido.
services/auth.js	C1	Responsabilidad de autenticación.
pages/login.html	C1 + C5	C1 funcionalidad; C5 estilos.
pages/dashboard.html	C1 + C5	C1 estructura/navegación; C5 UI.
js/candidatos.js + páginas	C2	C5 revisa UI sin cambiar lógica sin coordinar.
js/vacantes.js + páginas	C2	Misma regla.

Archivo / carpeta	Principal	Regla
js/empresas.js + páginas	C3	Misma regla.
js/postulaciones.js + páginas	C3	Misma regla.
js/entrevistas.js + páginas	C4	Misma regla.
js/tareas.js + páginas	C4	Misma regla.
css/*	C5	C1 coordina cambios estructurales si afectan integración.
docs/*	C5 coordina	Todos aportan información de su módulo.
README.md	C1 + C5	Todos entregan datos y evidencias.

5. Plan por fases

FASE 1 · DÍA 1 — PLANIFICACIÓN

Objetivo: dejar lista la arquitectura y el plan antes del desarrollo fuerte.

- C1: crear repositorio, ramas, estructura de carpetas y servicio base; definir flujo de autenticación.
- C2: investigar /users y /products; definir campos visibles y formularios.
- C3: investigar /carts y /posts; definir mapeo de negocio y campos.
- C4: investigar /comments y /todos; definir mapeo de negocio y campos.
- C5: proponer wireframe/estilo, matriz de pruebas y estructura de evidencias.
- Todos: usar NotebookLM para investigación sobre Fetch API, DummyJSON, PUT vs PATCH, headers, tokens, async/await y dominio de empleabilidad.
- Todos: registrar preguntas y respuestas útiles en la bitácora.

Entregable de Fase 1

- Documento de planificación.
- Tabla de módulos y endpoints.
- Requerimientos funcionales/no funcionales.
- Flujo de autenticación.
- Estructura inicial del repositorio.
- Estrategia de ramas.
- Resumen del dominio apoyado por NotebookLM.

FASE 2 · DÍA 2 — DESARROLLO

- C1 implementa login, token, protección y servicios compartidos.
- C2 implementa Candidatos y Vacantes.
- C3 implementa Empresas y Postulaciones.
- C4 implementa Entrevistas/Notas y Tareas.
- C5 implementa UI global y comienza pruebas de cada módulo conforme llega.
- Todos hacen commits frecuentes y documentan bloqueos/respuestas de NotebookLM.
- Cada módulo debe pasar una prueba funcional antes de integrarse.

Entregable de Fase 2

- Aplicación funcionando con login + seis módulos.
- Ramas y commits visibles en Git.
- Bitácora de dudas técnicas resueltas con NotebookLM.

FASE 3 · DÍA 3 — CIERRE Y ENTREGA

- C1 integra todas las ramas, resuelve conflictos y prepara la versión final.
- C2/C3/C4 validan nuevamente sus módulos ya integrados.
- C5 ejecuta pruebas de flujo completo y recopila evidencias.

- Todos aportan contenido al README y documento reflexivo.
- Equipo prepara video de presentación e infografía con apoyo de NotebookLM.
- Revisión final del repositorio, nombres, errores, enlaces, README y archivos entregables.

6. Estrategia Git para trabajar sin conflictos

Ramas recomendadas

```
main
integration
feature/login-auth
feature/candidatos-vacantes
feature/empresas-postulaciones
feature/entrevistas-tareas
feature/ui-testing
```

Flujo obligatorio recomendado

- Actualizar la rama base antes de comenzar.
- Trabajar en la rama propia.
- Hacer commits pequeños y descriptivos.
- Subir la rama al remoto.
- Crear Pull Request.
- C1 revisa integración; C5 puede revisar comportamiento/UI.
- Integrar primero a integration.
- Ejecutar pruebas.
- Solo después pasar a main.

```
feat(auth): implementar login con DummyJSON
feat(candidatos): implementar listado con GET
feat(vacantes): agregar creación con POST
feat(empresas): implementar eliminación
feat(postulaciones): agregar edición con PATCH
feat(ui): crear layout del dashboard
test(modulos): validar CRUD de candidatos
docs: actualizar bitácora de NotebookLM
```

Regla para evitar conflictos

- No editar deliberadamente archivos que pertenecen a otro compañero sin avisar.
- Los archivos compartidos (api.js, auth.js, dashboard.html, CSS global, README) requieren coordinación.
- Antes de un merge, el autor debe poder explicar qué cambió y cómo probarlo.
- No hacer trabajo directamente sobre main.

7. Plan de pruebas y criterios de aceptación

QA no debe esperar al último día. Cada integrante prueba su módulo y C5 consolida la matriz. La prueba final debe cubrir casos exitosos, errores de red, formularios inválidos y acceso sin autenticación.

Área	Pruebas mínimas	Responsable inicial	Validación final
Login	Credenciales válidas; inválidas; token; logout; acceso sin token	C1	C5
Candidatos	GET, POST, PUT, PATCH, DELETE, error de API	C2	C5
Vacantes	GET, POST, PUT, PATCH, DELETE, error de API	C2	C5

Área	Pruebas mínimas	Responsable inicial	Validación final
Empresas	GET, POST, PUT, DELETE, error de API	C3	C5
Postulaciones	GET, POST, PATCH, DELETE, error de API	C3	C5
Entrevistas	GET, POST, PATCH, DELETE, error de API	C4	C5
Tareas	GET, POST, PATCH, DELETE, error de API	C4	C5
UI	Navegación, responsive, mensajes, formularios y estados	C5	Todos
Integración	Login → dashboard → seis módulos → logout	C1	C5 + todos

Checklist de aceptación antes de entregar

- El login funciona con las credenciales de prueba del enunciado.
- El token se guarda y el acceso sin token queda restringido.
- Logout elimina el token.
- Los seis módulos son accesibles desde la interfaz principal.
- Cada módulo implementa exactamente los métodos exigidos.
- Todas las operaciones usan fetch y async/await.
- Las operaciones están dentro de manejo try/catch.
- Existen mensajes de éxito y error.
- La interfaz funciona en distintos tamaños de pantalla.
- No hay errores de JavaScript que rompan la aplicación.
- README.md está actualizado.
- Bitácora de NotebookLM contiene las principales dudas y soluciones.
- Video e infografía están listos.
- Git contiene ramas y commits de los cinco integrantes.

8. NotebookLM: cómo repartir su uso

El enunciado establece NotebookLM como herramienta obligatoria de investigación, consulta y apoyo para el video y la infografía. No debe utilizarse únicamente al final.

Fase	Uso de NotebookLM	Evidencia
Planificación	Investigar Fetch API, DummyJSON, reclutamiento, PUT/PATCH, headers y async/await.	Resumen de tareas
Desarrollo	Resolver bloqueos concretos: body de POST, protección sin token, manejo de errores de red, etc.	Bitácora de dudas
Cierre	Analizar código/documentación para apoyar README, video e infografía.	README, video, infografía y reflexión

Formato recomendado de bitácora

- Fecha.
- Integrante.
- Problema o pregunta.
- Consulta realizada en NotebookLM.

- Respuesta útil obtenida.
- Cómo se aplicó al proyecto.
- Resultado de la aplicación.

9. Entregables finales y responsables

Entregable	Responsable principal	Participación
Repositorio Git	C1	Todos
Aplicación frontend funcional	Todos	Todos
Login + autenticación	C1	C5 prueba
Candidatos + Vacantes	C2	C5 prueba
Empresas + Postulaciones	C3	C5 prueba
Entrevistas + Tareas	C4	C5 prueba
README.md	C1 + C5	Todos aportan
Bitácora NotebookLM	C5	Todos aportan
Documento reflexivo	C5	Todos aportan
Video	C5 coordina	Todos participan
Infografía	C5 coordina	Todos aportan/revisan

10. Cronograma operativo recomendado

Momento	C1	C2	C3	C4	C5
Inicio	Repo + arquitectura	Investiga users/products	Investiga carts/posts	Investiga comments/todos	Elabora QA plan
Planificación	Auth flow + API	Campos + CRUD	Campos + CRUD	Campos + CRUD	Wireframe + pruebas
Desarrollo 1	Login + services	Candidatos	Empresas	Entrevistas	CSS + dashboard
Desarrollo 2	Protección + integración	Vacantes	Postulaciones	Tareas	Pruebas parciales
Cierre	Merge + revisión	QA módulo 1/2	QA módulo 3/4	QA módulo 5/6	QA integral + docs
Entrega	Repo final	Evidencias	Evidencias	Evidencias	README/video/infografía

11. Riesgos y cómo evitarlos

Riesgo	Prevención
Dos personas modifican el mismo archivo	Asignar propiedad de archivos y coordinar cambios compartidos.
Integración rompe módulos	Integrar primero en integration y probar antes de main.
Se pierde tiempo al final haciendo documentación	Registrar evidencias y consultas de NotebookLM desde el Día 1.
Un CRUD funciona pero no cumple los requisitos	Usar los pedidos HTTP como checklist por módulo.
La app falla ante un error de API	try/catch, mensajes de error y pruebas negativas.
Se interpreta la falta de persistencia como error	Recordar que DummyJSON simula mutaciones y no persiste cambios.
Un integrante queda sin commits	Asignar tareas concretas y commits verificables desde el inicio.
Problemas de UI después de integrar	C5 prueba responsive y navegación en integration.

12. Orden recomendado para empezar

No comiencen creando los seis CRUD simultáneamente. Sigán este orden:

- 1. Crear repositorio y ramas.
- 2. Crear estructura de carpetas.
- 3. Crear base de API/service compartido.
- 4. Implementar y probar login.
- 5. Crear dashboard y navegación.
- 6. Desarrollar los seis CRUD en paralelo por equipos.
- 7. Integrar módulo por módulo.
- 8. Ejecutar pruebas completas.
- 9. Corregir errores.
- 10. Completar README, bitácora, reflexión, video e infografía.
- 11. Revisar historial Git y participación de los cinco.
- 12. Entregar.

Conclusión

La división propuesta cubre el proyecto completo y mantiene separadas las áreas con mayor riesgo de conflicto. C2, C3 y C4 pueden desarrollar sus dos módulos de forma paralela; C1 construye la base transversal y controla la integración; C5 mantiene la calidad visual, las pruebas y los entregables. Todos participan en planificación, desarrollo y cierre, respetando las condiciones del enunciado.

Referencia: Enunciado oficial “JobConnect — Segundo Mini Proyecto · Frontend”, 6 páginas, proporcionado para este trabajo.